第 4 章　テスト粒子で円盤銀河を作る

第 3 章では、固定した Plummer ポテンシャル中で 1 個の星を動かしました。この章では、同じ重力場の中に多数の星を配置し、回転する円盤を作ります。新しく学ぶ数値計算法は線形補間です。前章の台形則と乱数を組み合わせ、指定した分布に従う粒子の位置を生成します。時間発展には、引き続き leapfrog 法の kick–drift–kick（KDK）形式を使います。

ここで作るのは、自己重力で支え合う円盤ではありません。重力は Plummer 球が作り、星を表すテスト粒子はその重力を受けるだけです。粒子を多数に増やすことと、粒子同士の重力を計算することを区別します。まず一つの銀河の中で円盤が保たれることを確かめ、第 5 章の相互作用へ進む準備をします。

この章で学ぶこと

- 円盤の面密度と、粒子の半径の確率密度の違いを説明できます。
- 台形則で累積分布関数を作り、線形補間で逆変換できます。
- 乱数から位置を生成し、同じ Plummer モデルの円運動速度を与えられます。
- 1 粒子の加速度を NumPy で多数粒子へ拡張し、KDK で時間発展させられます。
- 分布表の誤差、有限粒子数のばらつき、軌道積分の誤差を分けて確認できます。

4.1 重力を作る球と表示する円盤を区別する

4.1.1 本章のモデルを定義する

原点に質量 M、スケール長 a の Plummer 球を固定します。ポテンシャルは前章と同じ

$$\Phi(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{4.1}$$

です。[2] 球の質量は時間によらず一定で、粒子の配置を変えても変わりません。質量をもたないテスト粒子を x–y 平面に置き、初期速度もその平面内で与えます。外部からの力、ガス、星形成、動力学的摩擦は含めません。

粒子 i の位置と速度は $\boldsymbol{r}_i = (x_i, y_i)$、$\boldsymbol{v}_i = (v_{x,i}, v_{y,i})$ です。円盤の中心からの半径を $R = \sqrt{x^2 + y^2}$ と書きます。重力を作る球の半径 $r = \sqrt{x^2 + y^2 + z^2}$ とは定義が異なりますが、本章では $z = 0$ なので $r = R$ です。R はここでは円盤の半径であり、後の銀河中心の位置ベクトル $\boldsymbol{R}_k$ ではありません。

4.1.2 円盤の粒子に質量を割り当てない

画面上に円盤が現れても、その粒子分布から重力場を計算しているわけではありません。Plummer 球の質量 M を N 等分して粒子へ配る操作も行いません。M は背景ポテンシャルのパラメータで、N は円盤を描くために追跡する粒子数です。

本章では円盤の形を**粒子数面密度** Σ_N で定義します。これは面積当たりの粒子数です。各粒子に同じ表示上の重みを与えれば恒星分布の形を表せますが、その重みを加速度計算へ入れることはしません。このため、円盤の分布と Plummer 球の密度は独立に選べます。自分自身の重力まで含めた平衡銀河を構成する問題とは、区別してください。

よくある失敗：多数の粒子があっても、相互重力はまだありません

各粒子は同じ解析的ポテンシャルから独立に力を受けます。粒子数を 2 倍にしても、すでに置いた 1 粒子への力は変わりません。本章は多数のテスト粒子の計算であり、自己重力円盤の N 体計算ではありません。

4.2 指数円盤を半径の確率分布へ直す

4.2.1 初期の配置範囲を決める

中心ほど粒子が多い円盤を作るため、初期の粒子数面密度を

$$\Sigma_N(R) = \Sigma_{N,0}\exp(-R/R_d), \qquad 0 \le R \le R_{\max} \tag{4.2}$$

とします。R_d は指数分布のスケール長、$R_{\max}$ は初期粒子を配置する外縁です。この形は構成案で選んだ教材用モデルです。本章では $R_d = a = 3\,\mathrm{kpc}$ としますが、二つは同じ物理量ではなく、等しくする必要もありません。

第 3 章の r_{tab} は、無限遠まで続く Plummer 球の質量表を作る範囲でした。一方、今回は初期粒子分布を $R_{\max}$ で切断すると定義しています。ただし、Plummer 球の密度や重力をそこで切断しません。また、時間積分中に $R_{\max}$ を越えた粒子を取り除いたり、跳ね返したりもしません。初期の配置範囲と、運動の境界条件は別です。

4.2.2 円環の面積を含める

半径 R から $R+dR$ までの薄い円環の面積は $2\pi R\,dR$ です。その中の期待粒子数は

$$dN = 2\pi R\Sigma_N(R)\,dR \tag{4.3}$$

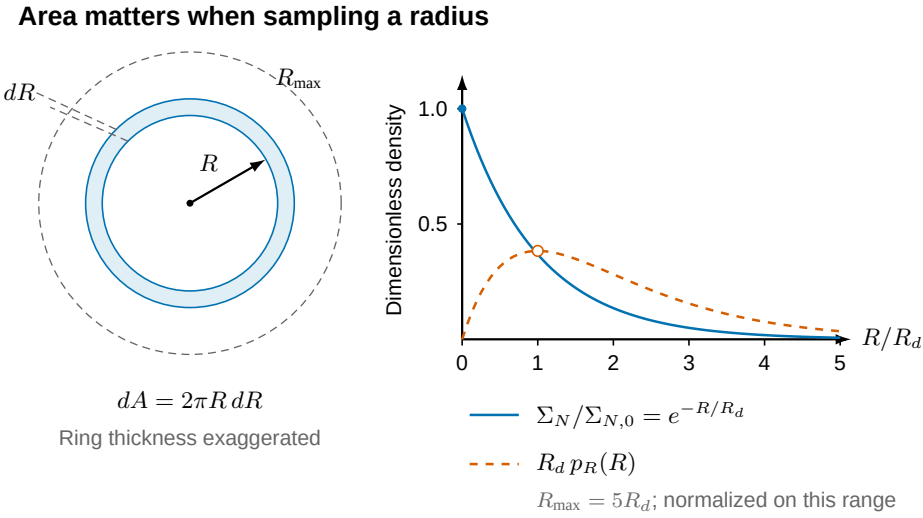
となります。全粒子数で割れば、半径の確率密度 $p_R(R)$ が得られます。

$$p_R(R) = \frac{2\pi R\Sigma_N(R)}{N} = \frac{R\exp(-R/R_d)}{\displaystyle\int_0^{R_{\max}} R'\exp(-R'/R_d)\,dR'}. \tag{4.4}$$

p_R の単位は長さの逆数です。$p_R(R)\,dR$ は無次元の確率であり、面密度とは異なります。第 3 章の球殻には $4\pi r^2$ を使いましたが、ここで粒子を配置するのは円盤なので $2\pi R$ です。

Σ_N は中心で最大ですが、$p_R(0) = 0$ です。中心に近い円環ほど面積が小さいからです。$R_{\max} > R_d$ なら、$R\exp(-R/R_d)$ を微分すると、半径の確率密度は $R = R_d$ で最大になります。中心に面密度のピークがあることと、半径のヒストグラムのピークが中心にあることは同じではありません。

図4.1では、円環の面積と二つの分布の違いを対応させています。第 3 章の球殻（図3.1）と比べ、今回は体積ではなく面積を掛けることを確認してください。



$$dN \quad = \quad \Sigma_N(R)\,dA \quad = \quad 2\pi R\,\Sigma_N(R)\,dR$$

図 4.1　円環の面積と半径の確率密度の関係。面密度に $2\pi R$ を掛けて半径の分布へ直します。右側は $R_{\max} = 5R_d$ で規格化した解析的な入力分布で、標本化した粒子の結果ではありません。面密度は中心で最大でも、半径の確率密度は中心で 0 です。

よくある失敗：半径を指数乱数にすればよい、とは限りません

$p_R \propto \exp(-R/R_d)$ としてしまうと、円環の面積に由来する R を落としてしまいます。また、R を一様乱数で選んでも、面積当たり一様な円盤にはなりません。何に対する密度を指定しているかを、最初に確認します。

4.3 台形則で累積分布関数を作る

4.3.1 半径以内に入る確率を求める

半径 R 以内に入る確率を、累積分布関数（cumulative distribution function; CDF）と呼びます。ここでは

$$F(R) = \int_0^R p_R(R')\,dR' = \frac{\displaystyle\int_0^R R' \exp(-R'/R_d)\,dR'}{\displaystyle\int_0^{R_{\max}} R' \exp(-R'/R_d)\,dR'} \tag{4.5}$$

です。$F(0) = 0$、$F(R_{\max}) = 1$ で、その間で単調に増えます。累積分布とその逆関数から目的の分布に従う乱数を作る考え方は、文献 [4] の第 23 章で扱われています。

解析的な比較値も用意できます。$x = R/R_d$、$x_{\max} = R_{\max}/R_d$ として

$$C(x) \equiv 1 - (1 + x)e^{-x} \tag{4.6}$$

と置きます。$R' \exp(-R'/R_d)$ を積分すると $R_d^2 C(R/R_d)$ になるので、

$$F(R) = \frac{C(R/R_d)}{C(x_{\max})}, \qquad p_R(R) = \frac{R \exp(-R/R_d)}{R_d^2 C(x_{\max})} \tag{4.7}$$

です。初期分布の規格化定数は $\Sigma_{N,0} = N/[2\pi R_d^2 C(x_{\max})]$ です。この式は本章の面密度を積分して得た比較用の式であり、数値 CDF を作る処理には使いません。

$x_{\max} = 5$ では $C(x_{\max}) = 0.959572318$ です。無限に延ばした同じ指数分布のうち、約 4.04% が $5R_d$ より外側にあります。本章ではそこを初期配置の対象から外し、内側の条件付き分布を確率 1 へ規格化します。Plummer 球の $5a$ 以内の質量割合とは別の数です。

4.3.2 前章の累積積分を再利用する

分点を R_j、被積分関数を $g_j = R_j \exp(-R_j/R_d)$ として、台形の面積を順に足します。

$$I_0 = 0, \qquad I_{j+1} = I_j + \frac{g_j + g_{j+1}}{2}(R_{j+1} - R_j), \tag{4.8}$$

$$\widehat{F}_j = \frac{I_j}{I_{N_R}}. \tag{4.9}$$

N_R は区間数で、分点数は $N_R + 1$ です。ハットは数値近似を表します。I_j の単位は長さの 2 乗ですが、最後の値で割った $\widehat{F}_j$ は無次元です。

第 3 章では、有限範囲の累積質量を無条件に全質量 M へそろえませんでした。ここで最後の値で割ってよいのは、式(4.5)そのものが有限範囲で規格化した確率だからです。表を都合よく補正する操作ではありません。

```
def cumulative_trapezoid(x, values):
    """ 第 3 章と同じ、各分点までの台形の和です。"""
    x, values = check_grid(x, values)
    integral = np.zeros_like(x)
    for j in range(len(x) - 1):
        area = 0.5 * (values[j] + values[j + 1]) * (x[j + 1] - x[j])
        integral[j + 1] = integral[j] + area
    return integral
```

4.3.3 標準条件で CDF 表を実装する

内部単位は kpc、Gyr、$M_\odot$ です。速度は内部では $\mathrm{kpc\,Gyr^{-1}}$ を使い、表示するときだけ $\mathrm{km\,s^{-1}}$ へ換算します。質量・Plummer スケール長は第 3 章と共通です。その他の条件は、この章の孤立円盤を確認するために設定しました。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass, a = 1.0e11, 3.0                # Plummer 球の質量 [Msun] とスケール長 [kpc]
rd, rmax = 3.0, 15.0                 # 粒子分布のスケール長と初期外縁 [kpc]
n_particles = 1000
n_cdf = 4096                        # CDF 表の区間数。分点数は 4097 です。
seed, spin = 42, +1                  # +1 は反時計回り、-1 は時計回りです。
t_end, n_steps = 0.60, 3000          # [Gyr]。dt = 0.2 Myr です。
save_every = 10                      # 時間刻みではなく、履歴の保存間隔です。
dt = t_end / n_steps
```

表 4.1 第 4 章の標準条件。N_R、N、N_{step} はそれぞれ別の個数です。R_{max} は Plummer 球の外縁ではありません。

量	設定
Plummer 球	$M = 10^{11} M_\odot$, $a = 3\,\mathrm{kpc}$
粒子の初期分布	$R_d = 3\,\mathrm{kpc}$, $R_{\mathrm{max}} = 15\,\mathrm{kpc} = 5R_d$
粒子数・乱数 seed	$N = 1000$, seed $= 42$
CDF の区間数・分点数	$N_R = 4096$, 4097 分点
回転方向	$s_{\mathrm{rot}} = +1$（反時計回り）
終時刻・ステップ数	$t_{\mathrm{end}} = 0.60\,\mathrm{Gyr}$, $N_{\mathrm{step}} = 3000$
時間刻み・保存間隔	$\Delta t = 0.2\,\mathrm{Myr}$、10 ステップごと

```
def build_radial_cdf(rd, rmax, n_panels):
    check_positive(rd, "rd")
    check_positive(rmax, "rmax")
    check_integer(n_panels, "n_panels")
    radius_grid = np.linspace(0.0, rmax, n_panels + 1)
    weight = radius_grid * np.exp(-radius_grid / rd)
    cumulative = cumulative_trapezoid(radius_grid, weight)
    if not np.isfinite(cumulative[-1]) or cumulative[-1] <= 0.0:
        raise ValueError("The normalization must be positive and finite.")
    cdf = cumulative / cumulative[-1]
    if np.any(np.diff(cdf) <= 0.0):
        raise ValueError("The inverse CDF needs strictly increasing values; revise the
        ↪  grid/range.")
    return radius_grid, cdf
```

`check_positive` と `check_integer` は、正で有限な値、正の整数であることを確認する補助関数です。完全なコードは Notebook に含めます。`cdf` を後で逆変換するため、隣接する値がすべて増えていることも確認します。標準条件では、$R = 0$ の被積分関数が 0 でも最初の台形の面積は正なので、CDF の最初の区間も増えます。

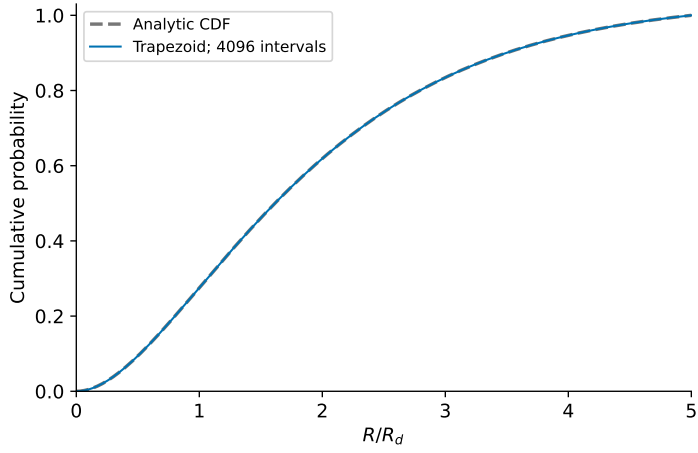


図 4.2 初期円盤の CDF。灰色破線は式(4.7)、青実線は台形則による数値表です。有限の初期配置範囲 $[0, 5R_d]$ で規格化しています。

分点上で解析式と比較した最大差は 9.29×10^{-8} でした。図4.2ではほぼ重なりますが、この近似誤差が 0 という意味ではありません。

4.4 線形補間で分点間の値を求める

4.4.1 2 点間の関数を直線で近似する

CDF 表は、決めた分点の値しか持ちません。分点の間の値を求めるため、二つの点を結ぶ直線で近似します。この操作が線形補間です。前章の台形則では同じ直線の下の面積を求めましたが、ここでは直線上の値そのものを使います。

一般の表 (x_j, y_j) について、$x_j \le x \le x_{j+1}$ の位置を

$$\lambda = \frac{x - x_j}{x_{j+1} - x_j} \tag{4.10}$$

で表します。左端なら $\lambda = 0$、右端なら $\lambda = 1$ です。二つの値をその割合で混ぜると、

$$y_{\mathrm{lin}}(x) = (1 - \lambda) y_j + \lambda y_{j+1} \tag{4.11}$$

となります。分点では表の値に一致し、その間では両端の値の間を取ります。傾きを用いた $y_j + (y_{j+1} - y_j)(x - x_j)/(x_{j+1} - x_j)$ とも同じ式です。補間と補間誤差の一般的な考え方は文献 [3] の §2.3 を参照してください。

4.4.2 一つの点を実装して確かめる

まず区間を順に探す、短い実装を作ります。区間が見つかったら、式(4.10)と式(4.11)を計算するだけです。入力する分点は狭義単調増加とします。

```
def linear_interp_one(x, xp, fp):
    """ 一つの点を補間します。本文 eq:ch4-linear-interpolation。"""
    xp, fp = check_grid(xp, fp)
    if not np.isfinite(x) or x < xp[0] or x > xp[-1]:
        raise ValueError("Interpolate only within the finite table range.")
    if x == xp[-1]:
        return float(fp[-1])
    for j in range(len(xp) - 1):
        if xp[j] <= x <= xp[j + 1]:
            fraction = (x - xp[j]) / (xp[j + 1] - xp[j])
            return (1.0 - fraction) * fp[j] + fraction * fp[j + 1]
    raise RuntimeError("No interpolation interval was found.")
```

この関数は表の外では停止します。補間は表の内側での近似であり、外側へ直線を延ばす外挿とは区別します。`check_grid` は第 3 章と同じく、分点・値の配列の形と有限性、分点の増加を確かめます。

多数の点を補間するときには、`np.interp(x, xp, fp)` を使います。`xp` が入力側の分点、`fp` がその値です。自作関数で原理を確認したうえで、同じ区分的な線形補間を行うライブラリ関数に置き換えます。[9] 全粒子について Python で表を繰り返し走査する必要はありません。

よくある失敗：増加していない表をそのまま渡さない

入力側の分点に重複があると、式(4.10)の分母が 0 になります。表を作る段階で、分点が有限で狭義単調増加かを確認します。また、`np.interp` は既定で

は表の外で端の値を返します。本章の自作関数とはこの扱いが違うので、呼び出し前に補間点が表の範囲内にあることを確認します。

4.4.3 線形補間の精度と表の誤差を区別する

真の関数 f が滑らかで、両端の値が正しく与えられている場合、線形補間の誤差は区間内のある点 ξ を使って

$$f(x) - f_{\mathrm{lin}}(x) = \frac{f''(\xi)}{2}(x - x_j)(x - x_{j+1}) \tag{4.12}$$

と書けます。区間幅を h とし、$|f''|$ が上から抑えられるなら、誤差の大きさは $\max|f''|\,h^2/8$ 以下です。これが、滑らかな関数を細かい表で線形補間したとき、通常は幅の 2 乗で誤差が減る理由です。式は多項式補間の誤差公式の 2 分点の場合です。[3]

ただし、これから使う CDF 表の値には台形則の誤差もあります。補間してもその誤差は消えません。さらに、CDF の逆関数は中心近くで傾きが大きくなるため、半径の誤差が全域で単純な同じ比例関係に従うとは限りません。本章では解析 CDF に戻した確率のずれを、まず確認します。

もう少し詳しく：ほかの補間法

多項式補間や三次スプラインは、より多くの分点を使って関数を近似します。高次の方法が有利な場合もありますが、点の間で振動したり、単調性を失ったりしないか確認が必要です。PCHIP は単調性を保つ区分的三次補間の代表例です。[10] 本章では、増加する表を扱いやすく実装も短い線形補間に集中します。詳細な理論と代替法の実装は数値計算の教科書に譲ります。

4.5 一様乱数を円盤の半径へ変換する

4.5.1 CDF の逆向きに表を読む

$0 \le u < 1$ の一様乱数を一つ選び、$F(R) = u$ となる半径を求めます。CDF は「半径から確率」を返しますが、ここでは「確率から半径」を求めます。この方法を逆変換サンプリングと呼びます。[4]

なぜこれで正しい分布になるのでしょうか。単調増加する F では、生成半径が R_* 以下になることと、一様乱数が $F(R_*)$ 以下になることが同値です。一様乱数がその範囲へ入る確率は $F(R_*)$ です。このため、生成半径の累積分布は目標の F になります。実装では、これを数値表の逆補間で近似します。

$\widehat{F}_j \le u \le \widehat{F}_{j+1}$ なら、入力側を $\widehat{F}$、出力側を R として

$$\widehat{R}(u) = R_j + \frac{u - \widehat{F}_j}{\widehat{F}_{j+1} - \widehat{F}_j}(R_{j+1} - R_j) \tag{4.13}$$

です。$\widehat{F}_j$ が増えているので、$\widehat{R}$ は必ず二つの分点の間に入ります。

```
# F -> R と表の役割を交換します。cdf は狭義単調増加でなければなりません。
u_demo = 0.5
radius_demo = linear_interp_one(u_demo, cdf, radius_grid)
radius_numpy = np.interp(u_demo, cdf, radius_grid)
print(f"Radius at CDF=0.5: manual={radius_demo:.9f}, NumPy={radius_numpy:.9f} kpc")
```

標準表で $u = 0.5$ を逆変換すると、半径は $4.843929332\,\mathrm{kpc}$ でした。自作関数と `np.interp` の値が一致することを確認します。これは初期円盤の粒子数の半分を含む半径で、Plummer 球の半質量半径ではありません。

図4.3の矢印を、縦軸の確率から横軸の半径へたどってください。

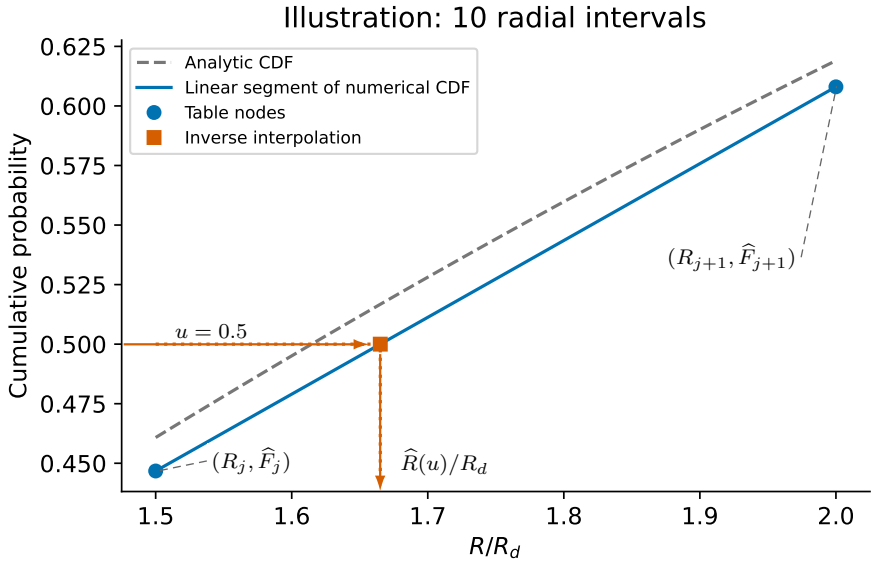


図 4.3 数値 CDF の一つの線分を逆向きに読む例。説明用に 10 区間だけで作った粗い表を使っています。丸が分点、四角が $u = 0.5$ の逆補間です。灰色破線の解析 CDF との差も示しています。標準の 4096 区間の結果とは区別してください。朱色の矢印は u から線分へ、そこから $\widehat{R}(u)$ へ読む順序です。表の組 $(R_j, \widehat{F}_j)$ を示し、横軸では R を R_d で規格化しています。

4.5.2 乱数を使う前に、逆変換を検証する

同じ数値表で $u \to \widehat{R} \to \widehat{F}$ と戻せば、線形補間の範囲ではもとの u にほぼ戻ります。しかし、それだけでは数値表が正しいかは分かりません。比較には、独立な解析 CDF である式(4.7)を使います。

まず分点上の誤差 $e_{F,\mathrm{node}}$ を、次に固定した確率の集合 $\mathcal{U}$ 上の逆変換の誤差 e_u を

$$e_{F,\mathrm{node}} = \max_j |\widehat{F}_j - F(R_j)|, \tag{4.14}$$

$$e_u = \max_{u \in \mathcal{U}} |F(\widehat{R}(u)) - u| \tag{4.15}$$

で定義します。本章の u は 0 から 1 までの 20001 個の等間隔の点です。まだ乱数は使いません。e_u は選んだ試験点上の最大値であって、連続な全確率範囲の最大誤差を厳密に保証する値ではありません。

```
u_check = np.linspace(0.0, 1.0, 20001)
cdf_rows = []
for count in [32, 64, 128, 256, 512]:
    nodes, values = build_radial_cdf(rd, rmax, count)
    node_error = np.max(np.abs(values - disk_cdf_exact(nodes, rd, rmax)))
    inverted = np.interp(u_check, values, nodes)
    probability_error = np.max(np.abs(disk_cdf_exact(inverted, rd, rmax) - u_check))
    cdf_rows.append([count, rmax / count, node_error, probability_error])
cdf_convergence = np.asarray(cdf_rows)
cdf_orders = np.log2(cdf_convergence[:-1, 3] / cdf_convergence[1:, 3])
```

表 4.2　CDF の分割数試験。範囲 $[0, 5R_d]$ と試験確率は固定しています。p_u は隣接する行の e_u の比から求めた推定次数です。

N_R	ΔR [kpc]	$e_{F,\mathrm{node}}$	e_u	p_u
32	0.468750	1.52×10^{-3}	2.45×10^{-3}	—
64	0.234375	3.80×10^{-4}	6.98×10^{-4}	1.8117
128	0.117188	9.51×10^{-5}	1.85×10^{-4}	1.9121
256	0.058594	2.38×10^{-5}	4.80×10^{-5}	1.9486
512	0.029297	5.94×10^{-6}	1.12×10^{-5}	2.0986

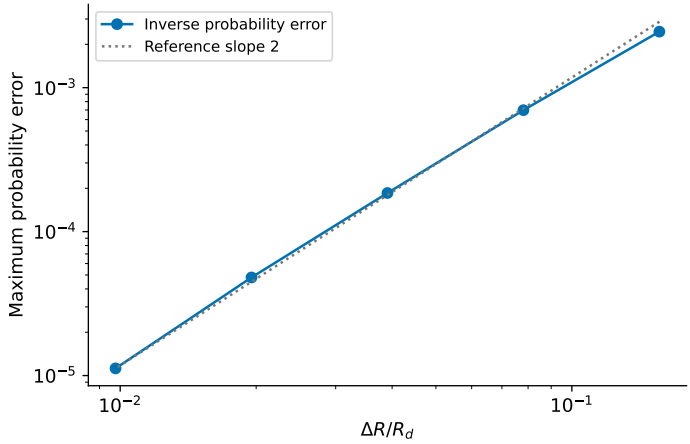


図 4.4　数値 CDF と逆補間による確率のずれ。点線は最も細かい分割の値を通る傾き 2 の参照線で、当てはめではありません。試験確率が有限個なので、最大誤差の位置の拾い方も推定次数に影響します。

表4.2では、分割を細かくすると誤差が減っています。標準の 4096 区間での e_u は 1.76×10^{-7} でした。この数値誤差と、次に見る有限粒子数のばらつきを分けて

考えます。表を細かくするだけで、乱数で配置した点が完全になめらかな分布になるわけではありません。

4.5.3 半径と方位角に乱数を与える

半径のための u_i とは別に、一様乱数 w_i を使って $\phi_i = 2\pi w_i$ とします。ここで ϕ は方位角です。二つを独立に選ぶことで、半径によらず方位角を一様にできます。位置は

$$x_i = R_i \cos\phi_i, \qquad y_i = R_i \sin\phi_i \tag{4.16}$$

です。同じ乱数一つを半径と角度に流用すると両者が関連し、意図した軸対称の分布を作れません。

Notebook では `rng = np.random.default_rng(seed)` で乱数生成器を用意します。`rng.random((N, 2))` の各行に、半径と角度に使う二つの一様乱数を入れます。seed は実験を再現するための設定です。同じ実装・乱数生成環境・呼び出し順序なら同じ初期値を再現できますが、seed だけで異なる実装や将来のライブラリ環境まで同一の乱数列になると保証はしません。[9] 配布物には初期配列と使用した NumPy の版も記録します。

もう少し詳しく：ほかの標本化法

棄却法は、候補の乱数を生成し、目標分布に応じて採用するかを選ぶ方法です。逆関数を求めにくい場合にも使えます。本書では、既習の台形則と今回の補間を一つの計算へ結び付けるため、CDF の逆変換を使います。乱数による積分を含む Monte Carlo 法の一般論は、文献 [4] に譲ります。

4.6 円運動速度を与えて円盤を作る

4.6.1 同じ重力場から初期速度を求める

粒子を置くだけでは、円盤を回転させたことにはなりません。各半径で Plummer 球の円運動速度を与えます。第 3 章の式を再利用すると、

$$v_c^2(R) = \frac{GMR^2}{(R^2 + a^2)^{3/2}} \tag{4.17}$$

です。粒子の分布のスケール長 R_d を、この分母の a へ取り違えないようにします。位置の確率を決める式と、重力を決める式は別です。

反時計回りの接線方向の単位ベクトルは $(-\sin\phi, \cos\phi)$ です。回転方向を $s_{\mathrm{rot}} = +1$ または -1 として、

$$v_{x,i} = -s_{\mathrm{rot}} v_c(R_i) \sin\phi_i, \qquad v_{y,i} = s_{\mathrm{rot}} v_c(R_i) \cos\phi_i \tag{4.18}$$

と置きます。例えば正の x 軸上の粒子は、$s_{\mathrm{rot}} = +1$ なら正の y 方向へ動き始めます。すべての粒子に同じ速度の大きさを与えるのではなく、それぞれの R_i に対応する v_c を使います。

本章の孤立円盤には、もう一つの銀河の軌道はありません。そのため、ここでは反時計回り・時計回りと呼びます。遭遇軌道の向きとの関係で定義する prograde・retrograde は、第 5・6 章で扱います。

```
def circular_speed_plummer(radius, mass, a):
    radius = np.asarray(radius, dtype=float)
    return np.sqrt(G * mass * radius**2 / (radius**2 + a**2)**1.5)


def potential_plummer(radius, mass, a):
    return -G * mass / np.sqrt(np.asarray(radius)**2 + a**2)
```

4.6.2 位置と速度を一つの初期配列へまとめる

半径の標本化、方位角、接線速度を一つの関数にまとめます。位置と速度はそれぞれ形 $(N, 2)$ です。最後に横に並べ、各行に $[x, y, v_x, v_y]$ を持つ形 $(N, 4)$ の状態配列を返します。

```
def make_disk(n_particles, mass, a, rd, rmax, spin, rng, n_panels=4096):
    check_integer(n_particles, "n_particles")
    check_positive(mass, "mass")
    check_positive(a, "a")
    if isinstance(spin, (bool, np.bool_)) or spin not in (-1, +1):
        raise ValueError("spin must be +1 or -1.")
    nodes, values = build_radial_cdf(rd, rmax, n_panels)
    uniform = rng.random((n_particles, 2))
    radius = np.interp(uniform[:, 0], values, nodes)
    phi = 2.0 * np.pi * uniform[:, 1]
    vc = circular_speed_plummer(radius, mass, a)
    pos = np.column_stack((radius * np.cos(phi), radius * np.sin(phi)))
    vel = spin * np.column_stack((-vc * np.sin(phi), vc * np.cos(phi)))
    return np.column_stack((pos, vel)), radius, phi
```

標準例では、`rng = np.random.default_rng(seed)` を用意して `make_disk` に渡します。乱数生成器を関数の中で毎回同じ seed へ戻しません。比較実験でまったく同じ初期標本を使う場合だけ、同じ seed の生成器を改めて作ります。初期粒子の半径は、今回の標本では約 0.100153 から 14.950926 kpc でした。上限 15 kpc 以内であることを確認します。

4.6.3 標本の重心を無条件に原点へ戻さない

有限個の粒子を乱数で置けば、単純な平均位置が原点に厳密には一致しないことがあります。しかし、ここで重要なのは、各粒子の位置と速度が固定した *Plummer* 中心に対する円軌道になっていることです。

粒子の平均位置だけを引くと、各粒子の Plummer 中心からの距離が変わります。平均速度だけを引いても、円運動速度との対応が変わります。第 2 章の有限質量 2 体系を重心系へ移す操作と混同せず、本章では粒子だけを再中心化しません。原点を移すなら、背景ポテンシャルの中心も含めた座標変換を考える必要があります。

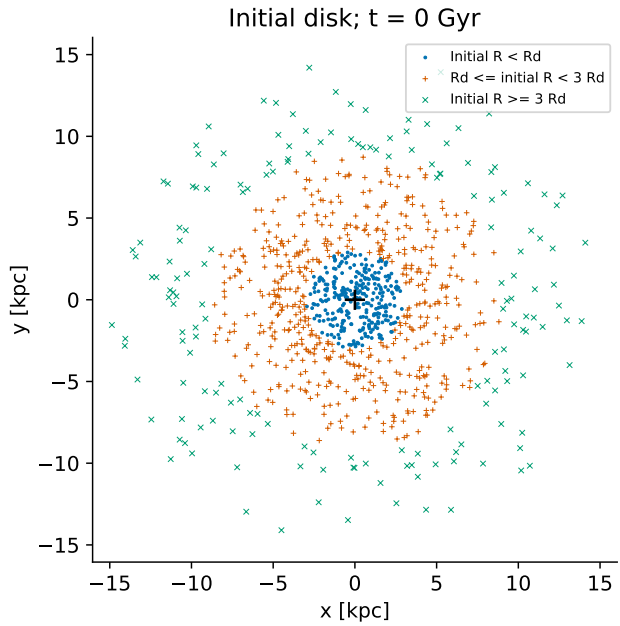


図 4.5　標準の初期円盤。初期半径による 3 群を色とマーカーで分けました。この分類は後の時刻でも固定します。中心の黒い十字は Plummer ポテンシャルの中心で、追加の質点ではありません。

4.7 生成した分布を確かめる

4.7.1 累積分布で比べる

円盤らしく見えることだけでは、式(4.2)に従っているか判断できません。粒子半径を小さい順に並べ、

$$F_N(R) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(R_i \le R) \tag{4.19}$$

を作ります。**1** は条件が真なら 1、偽なら 0 です。これは、その半径までにある粒子の割合を返す**経験 CDF** です。

有限個の標本なので、F_N は幅 $1/N$ の階段になります。連続的な解析 CDF に完全には一致しません。図4.6では、この階段と解析式を比較します。

ずれの指標を $D_N = \sup_R |F_N(R) - F(R)|$ とします。並べた半径 $R_{(j)}$ における階段の直前と直後、つまり $(j-1)/N$ と j/N を両方比べれば計算できます。

$$D_N = \max_{1 \le j \le N} \left\{ \left| \frac{j}{N} - F(R_{(j)}) \right|, \left| \frac{j-1}{N} - F(R_{(j)}) \right| \right\}. \tag{4.20}$$

本章では分布のずれの大きさとして使い、統計的な仮説検定の合否へ直結させません。

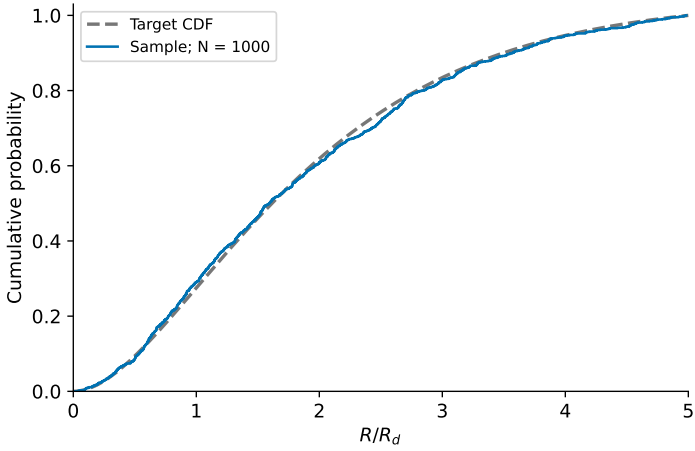


図 4.6　1000 粒子の経験 CDF と目標 CDF。数値 CDF 表の曲線ではなく、実際に生成した粒子から計算した割合です。有限標本のため、解析式の上下にずれます。

```
def empirical_cdf_distance(radius, rd, rmax):
    sorted_radius = np.sort(np.asarray(radius))
    n = len(sorted_radius)
    target = disk_cdf_exact(sorted_radius, rd, rmax)
    right = np.arange(1, n + 1) / n
    left = np.arange(n) / n
    return max(np.max(np.abs(right - target)), np.max(np.abs(left - target)))
```

標準標本では $D_N = 0.029414$ でした。これは数値表の逆変換誤差 1.76×10^{-7} よりずっと大きい値です。表をさらに細かくしても、この標本のばらつきが同じ割合で減るわけではありません。

4.7.2 円環から粒子数面密度を求める

円環の内側と外側の半径を $R_{j,-}, R_{j,+}$ とし、その中の粒子数を N_j とします。円環の正確な面積を使うと、全粒子数で規格化した数面密度の推定値は

$$\frac{\widehat{\Sigma}_{N,j}}{N} = \frac{N_j}{N\pi(R_{j,+}^2 - R_{j,-}^2)} \tag{4.21}$$

です。単位は kpc^{-2} です。区間の幅だけで割った半径のヒストグラムとは違います。

```
def surface_number_profile(radius, edges):
    """ 全粒子数で規格化した、円環ごとの数面密度 [kpc^-2]。"""
    counts, _ = np.histogram(radius, bins=edges)
    area = np.pi * (edges[1:]**2 - edges[:-1]**2)
    density = counts / (len(radius) * area)
    return density, counts, area
```

比較する解析値も、円環の中点の値だけでなく、同じ円環の平均

$$\left\langle \frac{\Sigma_N}{N} \right\rangle_j = \frac{F(R_{j,+}) - F(R_{j,-})}{\pi(R_{j,+}^2 - R_{j,-}^2)} \tag{4.22}$$

を使います。幅の広い円環では、点での値と平均値が異なるためです。時間発展後に、この同じ測り方で初期分布との比較も行います。

4.7.3 粒子数と seed を変える

CDF 表を固定したまま、$N = 250, 1000, 4000$ で標本化します。各 N について seed を 42 から 49 まで変え、8 回の D_N を比較した結果が表4.3です。これは同じ物理モデルを、異なる有限標本で表した比較です。

表 4.3 粒子数と seed による経験 CDF のずれ。平均・最小・最大は 8 個の seed での値です。数値積分法の収束次数を測った表ではありません。

N	D_N (seed 42)	平均	最小	最大
250	0.063392	0.052060	0.031292	0.066533
1000	0.029414	0.032565	0.017673	0.055907
4000	0.009382	0.012820	0.007856	0.018457

粒子数を増やすと、一般には分布のばらつきが小さくなります。ただし、個々の seed について D_N が必ず単調に減るとは限りません。ある領域に入る確率を p とすれば、独立な N 標本による粒子数の分散は $Np(1-p)$ です。割合のばらつきの尺度が $N^{-1/2}$ になることからも、粒子数と表の刻みは別の役割だと分かります。これは一つの領域の粒子割合に関する見積もりで、表の各 D_N が厳密に同じ係数で変わるという意味ではありません。

4.8 1 粒子の式を多数粒子へ拡張する

4.8.1 まず粒子ごとに計算する

Plummer 球の加速度は、第 3 章と同じ

$$\boldsymbol{a}_i = -\frac{GM\boldsymbol{r}_i}{(R_i^2 + a^2)^{3/2}} \tag{4.23}$$

です。新しいのは粒子番号 i が 1 個から N 個へ増える点だけです。1 粒子の成分計算を、まずループに入れます。

```
def acceleration_plummer_loop(pos, mass, a):
    acc = np.empty_like(pos)
    for i in range(len(pos)):
        x, y = pos[i]
        r2 = x**2 + y**2
        acc[i, 0] = -G * mass * x / (r2 + a**2)**1.5
        acc[i, 1] = -G * mass * y / (r2 + a**2)**1.5
    return acc
```

4.8.2 配列の軸を明示してまとめる

`pos` の形は $(N, 2)$ です。`np.sum(pos**2, axis=1)` は各粒子の $x_i^2 + y_i^2$ を返すので、`r2` の形は $(N,)$ になります。この分母を `[:, None]` で $(N, 1)$ へ変形し、各行の x, y 成分に同じ分母を使います。

```
def acceleration_plummer(pos, mass, a):
    """ 固定 Plummer 球から各粒子への加速度。本文 eq:ch4-acceleration。"""
    r2 = np.sum(pos**2, axis=1)
    acc = -G * mass * pos / (r2 + a**2)[:, None]**1.5
    return acc
```

`axis=0` で足すと、全粒子の x^2 の和と y^2 の和になり、別の量です。形が実行可能でも、意図した物理量になっているか確認します。独立なループ版との比較では、今回の差を GM/a^2 で規格化した最大値は 7.28×10^{-17} でした。演算の並べ方で末尾の桁が変わることはありますが、同じ加速度を計算しています。

この関数は $N \times N$ の距離表を作りません。背景の重力を粒子ごとに一度ずつ評価するため、1 回の加速度計算に必要な演算量は N に比例します。粒子対の重力を足す直接 N 体計算の $O(N^2)$ とは異なります。

よくある失敗：粒子番号と座標成分の軸を取り違えない

位置配列 $(N, 2)$ の第 1 軸は粒子、第 2 軸は成分です。`r2` は各粒子に一つの値を持ちます。分母を $(N, 1)$ にする操作は、同じ粒子の 2 成分へ値を対応させるためであり、粒子間相互作用を作る処理ではありません。

4.9 KDK で円盤を時間発展させる

4.9.1 積分器の計算は変えない

前章の KDK 関数は、位置と速度の配列へ加算・乗算する形でした。$(1, 2)$ を $(N, 2)$ へ増やしても、更新の 4 行は変わりません。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mass, a):
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mass, a)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

全粒子の最初の半 kick、全粒子の drift、新しい位置での加速度評価、最後の半 kick を順に行います。加速度を返し、次のステップへ再利用することも同じです。速度は最後の kick を終えた整数時刻の値を保存します。

4.9.2 時間刻みと出力間隔を分ける

多数粒子の位置と速度を毎ステップすべて保存すると、履歴のメモリが大きくなります。本章では時間積分は 3000 ステップすべて行いますが、履歴は 10 ステップごとに保存します。最後の時刻が保存間隔に乗らない場合も、最終状態は必ず保存します。

保存時刻の個数を N_{save} とすると、履歴の形は $(N_{\mathrm{save}}, N, 4)$ です。`float64` の各要素を 8 byte として、必要な領域は

$$B = 8 \times 4 \times N \times N_{\mathrm{save}} \quad \text{byte} \tag{4.24}$$

です。これは履歴配列だけの大きさで、一時配列や Notebook の画像のメモリは含みません。

表 4.4　標準の 1000 粒子・3000 ステップでの履歴配列。MB は 10^6 byte です。どちらも時間積分の力評価回数は 3001 回です。

保存方法	N_{save}	履歴の容量 [MB]
毎ステップ	3001	96.032
10 ステップごと	301	9.632

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mass, a, save_every=10):
    check_positive(mass, "mass")
    check_positive(a, "a")
    state, saved_steps, times, history, dt = prepare_history(state0, t_end, n_steps,
    ↪  save_every)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mass, a)
    output_index = 1
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mass, a)
        if not np.all(np.isfinite(pos)) or not np.all(np.isfinite(vel)):
            raise FloatingPointError("Non-finite state; check the model and time step.")
        if n + 1 == saved_steps[output_index]:
            history[output_index] = np.column_stack((pos, vel))
            output_index += 1
    return times, history
```

`prepare_history` は初期値の形 $(N, 4)$、有限性、正のステップ数などを確認し、保存するステップ番号と配列を用意します。詳しいコードは Notebook に置きます。ループは保存の有無と無関係に毎ステップ進みます。標準の時間刻みは 0.2 Myr、保存間隔は 2 Myr です。`history[j]` の j は保存番号であり、積分した全ステップの番号ではありません。

4.9.3 孤立円盤を 0.6 Gyr 進める

初期配列を作った後、`integrate_leapfrog` へ加速度関数、終時刻、ステップ数、M, a を渡します。標準では初回の加速度と 3000 回の更新で、合計 3001 回の加速度関数の呼び出しです。各呼び出しで 1000 粒子分をまとめて評価します。

中心近くの角速度は $\Omega_0 = \sqrt{GM/a^3}$ に近づきます。対応する周期は $2\pi\sqrt{a^3/(GM)} \simeq 48.677\,\mathrm{Myr}$ です。円盤全体で一つの周期があるわけではありません。中心側の短い時間尺度も意識して刻みを選び、後の刻み試験で確認します。

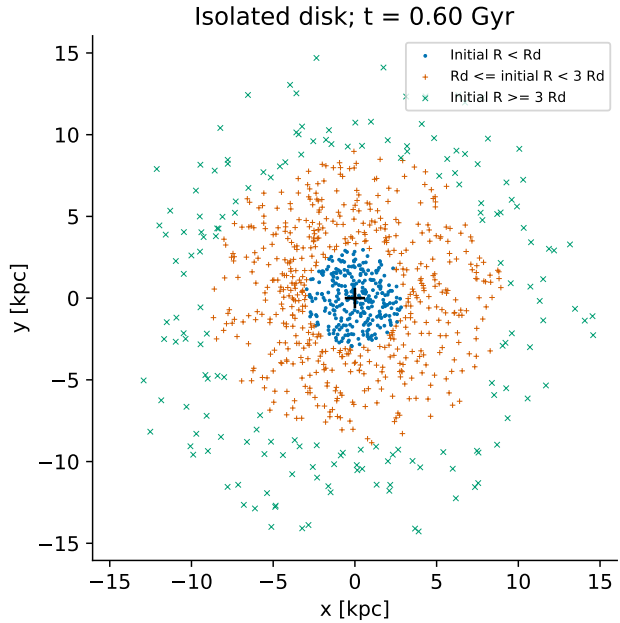


図 4.7 0.60 Gyr 後の孤立円盤。軸範囲、初期半径による色・マーカー分類は図4.5と同じです。各粒子の方位角は変わっていますが、半径方向の分布はほぼ保たれています。

円運動の初期条件では、連続的な軸対称の数面密度は時間に依存しません。ただし、角速度は半径によって異なるので、有限個の点の模様全体が一枚の板のように剛体回転するわけではありません。密度分布が定常であることと、個々の粒子が動かないことは別です。

4.10 分布の維持と軌道の精度を確認する

4.10.1 初期と終時刻の数面密度を比較する

初期と終時刻に同じ円環を用い、式(4.21)で数面密度を測ります。図4.8の灰色点線は、式(4.22)の円環平均です。初期標本からのばらつきが、時間積分中に新しく生じた数値誤差であるとは限りません。

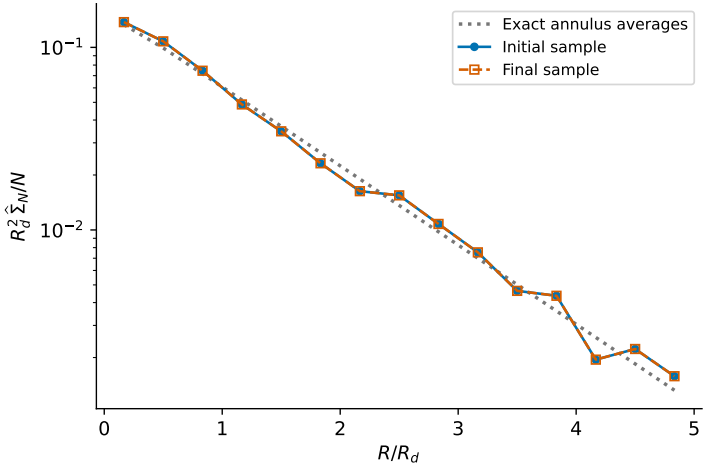


図 4.8 円環で測った数面密度。青丸が初期、朱色の白抜き四角が終時刻です。縦軸は $R_d^2\hat{\Sigma}_N/N$ です。粒子数の少ない外側では標本のばらつきが目立ちますが、初期と終時刻の曲線はほぼ重なります。

経験 CDF のずれは、初期で 0.029414、終時刻で 0.029420 でした。小さな違いはあっても、有限粒子数に由来する初期のずれが支配的です。ここで確かめているのは、固定ポテンシャル中の円軌道の集合が保たれることです。自己重力円盤の集団的な安定性を調べた結果とは解釈しません。

4.10.2 各粒子の保存量を計算する

固定した球対称場なので、各粒子の比エネルギーと比角運動量を使います。

$$\varepsilon_i = \frac{v_{x,i}^2 + v_{y,i}^2}{2} + \Phi(R_i), \qquad \ell_{z,i} = x_i v_{y,i} - y_i v_{x,i}. \tag{4.25}$$

ここでも粒子の質量を掛けません。Plummer 球の全エネルギーや、質量をもつ円盤全体のエネルギーを計算するのではありません。

```
def diagnostics(history, mass, a):
    pos, vel = history[:, :, :2], history[:, :, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=2))
    energy = 0.5 * np.sum(vel**2, axis=2) + potential_plummer(radius, mass, a)
    angular = pos[:, :, 0] * vel[:, :, 1] - pos[:, :, 1] * vel[:, :, 0]
    return radius, energy, angular
```

履歴は 3 軸なので、成分を足す軸は `axis=2` です。半径・エネルギー・角運動量の返り値は、いずれも (N_{save}, N) です。全粒子のエネルギー変化を先に足すと、正負の誤差が相殺されることがあるため、粒子ごとの誤差を調べます。

本章の円軌道では全粒子の $\varepsilon_i^0 < 0$ なので、エネルギー誤差は各初期値の絶対値で割れます。一方、中心に近い粒子では $\ell_{z,i}^0$ が小さくなります。角運動量には共通の正の尺度 $\ell_* = R_d v_c(R_d)$ を使い、

$$e_\varepsilon = \max_{n,i} \frac{|\varepsilon_i^n - \varepsilon_i^0|}{|\varepsilon_i^0|}, \qquad e_\ell = \max_{n,i} \frac{|\ell_{z,i}^n - \ell_{z,i}^0|}{\ell_*} \tag{4.26}$$

とします。この e_ℓ は、各粒子の初期角運動量に対する相対誤差ではありません。

4.10.3 初期条件から同時刻の解析位置を作る

円盤が円く見えることだけでは、回転の速さまで正しいと分かりません。本章では全粒子に円運動を与えたので、初期半径から

$$\Omega_i = \frac{\sqrt{GM}}{(R_i(0)^2 + a^2)^{3/4}} \tag{4.27}$$

を求められます。解析的な方位角は $\phi_i(t) = \phi_i(0) + s_{\mathrm{rot}}\Omega_i t$ です。この関係で初期位置を回転させ、同じ時刻の数値位置と比較します。比較の途中で数値解の半径から Ω_i を更新してはいけません。

```
def exact_disk_positions(times, state0, mass, a, spin):
    """ 円運動の初期条件にだけ使う、同時刻の解析位置です。"""
    pos0 = state0[:, :2]
    radius2 = np.sum(pos0**2, axis=1)
    omega = np.sqrt(G * mass) / (radius2 + a**2)**0.75
    angle = spin * np.asarray(times)[:, None] * omega[None, :]
    c, s = np.cos(angle), np.sin(angle)
    x = pos0[None, :, 0] * c - pos0[None, :, 1] * s
    y = pos0[None, :, 0] * s + pos0[None, :, 1] * c
    return np.stack((x, y), axis=2)
```

式(4.27)は原点でも有限です。原点にちょうど置いた粒子は初期位置が 0 なので、この回転式でもそのまま 0 にとどまります。v_c/R をそのまま数値的に評価する場合の $0/0$ を作りません。

位置の誤差と半径だけの変化は、別に定義します。

$$e_{\mathrm{pos}} = \max_{n,i} \frac{|\boldsymbol{r}_i^n - \boldsymbol{r}_{i,\mathrm{exact}}(t_n)|}{R_d}, \qquad e_R = \max_{n,i} \frac{|R_i^n - R_i^0|}{R_d}. \tag{4.28}$$

すべての粒子に共通の R_d で規格化します。n は、ここでは検証に使う保存時刻を走る添字です。したがって、これらは保存時刻における最大値であり、保存していない時刻の最大誤差まで保証するものではありません。

```
def disk_error_metrics(times, history, state0, mass, a, rd, spin):
    radius, energy, angular = diagnostics(history, mass, a)
    exact = exact_disk_positions(times, state0, mass, a, spin)
    position_error = np.max(np.linalg.norm(history[:, :, :2] - exact, axis=2)) / rd
    radius_error = np.max(np.abs(radius - radius[0])) / rd
    energy_error = np.max(np.abs(energy - energy[0]) / np.abs(energy[0]))
    angular_scale = rd * circular_speed_plummer(rd, mass, a)
    angular_error = np.max(np.abs(angular - angular[0])) / angular_scale
    return np.array([position_error, radius_error, energy_error, angular_error])
```

表 4.5 標準の 1000 粒子・0.60 Gyr・3000 ステップの確認。301 個の保存時刻と全粒子について最大値を取りました。規格化は式(4.26)、式(4.28)です。

指標	値
位置 e_{pos}	5.46×10^{-4}
半径 e_R	4.73×10^{-5}
比エネルギー e_ε	2.19×10^{-9}
比角運動量 e_ℓ	5.00×10^{-14}

位置誤差は 5.46×10^{-4}、半径の変化は 4.73×10^{-5} でした。比エネルギーの相対誤差は 2.19×10^{-9} で、角運動量のずれは 5.00×10^{-14} です。中心力の kick と drift が角運動量を保つ性質は、各粒子にそのまま成り立ちます。一方、エネルギー差や位相の差は有限刻みで残ります。表4.5は本章の実測値で、任意の円盤・刻みで同じ値になるという主張ではありません。

4.11 数値実験の条件を一つずつ変える

4.11.1 時間刻みだけを変える

初期配列 `state0` を保存し、同じ終時刻 0.60 Gyr まで 750、1500、3000、6000 ステップで進めます。この比較では乱数を引き直しません。また、各計算の保存時刻を 151 個の同じ時刻にそろえ、式(4.28)を比べます。

```
orbit_rows = []
for count in [750, 1500, 3000, 6000]:
    t_check, h_check = integrate_leapfrog(acceleration_plummer, state0, t_end, count,
    ↪  mass, a, count // 150)
    errors = disk_error_metrics(t_check, h_check, state0, mass, a, rd, spin)
    orbit_rows.append([count, t_end / count, *errors])
orbit_convergence = np.asarray(orbit_rows)
orbit_orders = np.log2(orbit_convergence[:-1, 2] / orbit_convergence[1:, 2])
```

最も細かい側の推定次数は 2.00004 となり、2 次精度を確認できます。保存量が良好でも、同じ時刻の位置を独立に確かめることを続けます。円運動速度を変更した非円軌道の実験には、この円軌道の解析位置を使いません。

表 4.6　同じ初期粒子を使った KDK の時間刻み試験。全行で終時刻と 151 個の評価時刻をそろえています。p は隣接する行の e_{pos} から求めた推定次数です。

N_{step}	Δt [Myr]	e_{pos}	p	力評価回数
750	0.8000	8.74×10^{-3}	—	751
1500	0.4000	2.18×10^{-3}	2.00069	1501
3000	0.2000	5.46×10^{-4}	2.00017	3001
6000	0.1000	1.36×10^{-4}	2.00004	6001

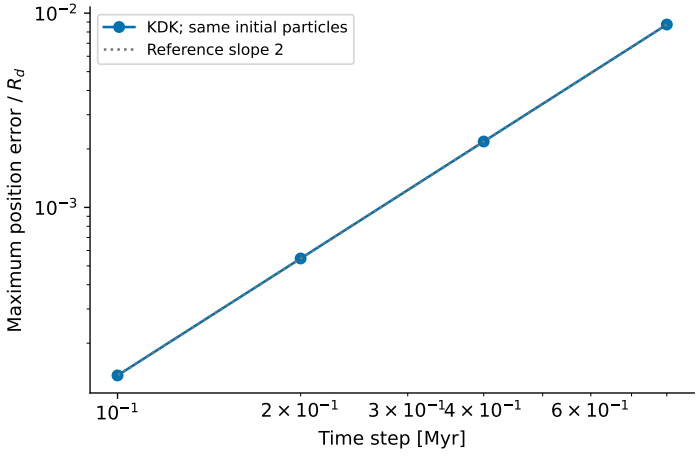


図 4.9　時間刻みと最大位置誤差。点線は最も細かい刻みの点を通る傾き 2 の参照線です。粒子数や CDF 表ではなく、軌道を進める刻みだけを変更しています。

4.11.2 粒子数・CDF 表・保存間隔の役割を区別する

本章には複数の「数」が出てきました。N_R を増やすと CDF の積分・逆補間の精度が変わります。N を増やすと分布を表す標本が増えます。N_{step} を増やすと、同じ終時刻へ進む時間刻みが細かくなります。`save_every` を変えると、履歴の容量と、保存された時刻で見つけられる変動が変わります。

保存の頻度だけを変えても、同じ初期値・同じ KDK 更新なら終点は変わりません。ただし、粗い出力では短い時間の変動や誤差のピークを見逃すことがあります。「図がなめらかになること」と「運動方程式を高精度で解くこと」を混同しないようにします。

4.11.3 回転方向を逆にする

同じ初期半径・方位角で $s_{\mathrm{rot}} = -1$ にすれば、初期位置は同じ、速度は逆になります。比エネルギーは同じで、角運動量は符号を反転します。固定した球対称場では、どちらの向きでも円軌道を作れます。本章の再計算でも、時計回りの解析位置に対する誤差は 5.46×10^{-4} でした。

一般の乱数配置について、初期位置をそのままに速度だけを反転しても、円盤のすべての点が同じ時刻に x 軸の鏡映になるわけではありません。対応するのは同じ初期位置から時間を逆向きにたどる運動です。個々の粒子は動いても、連続的な軸対称分布は両方で同じです。

4.11.4 静止画とアニメーションを比較する

Notebook では、保存済みの履歴から円盤のアニメーションを作ります。全粒子のうち 3 個だけを別のマーカーで追い、星が実際に動いていることも見られるようにしています。これらの粒子に特別な質量や力を与えてはいません。

再生間隔は画面表示の速度です。物理的な Δt とも、履歴の保存間隔とも異なります。動画を見るだけで検証を終えず、初期・終時刻の静止画、数面密度、保存量、位置誤差も残します。

4.12 本章のまとめ

本章では、静的な Plummer 球の中で回転するテスト粒子円盤を作りました。円盤の数面密度に円環の面積を掛けると、半径の確率密度が得られます。台形則で累積分布関数を作り、線形補間でその逆関係を読み、一様乱数を半径へ変換しました。方位角には独立な乱数を使います。

Plummer 球の円運動速度を半径ごとに与え、1 粒子の加速度を多数粒子の配列へ拡張しました。KDK の更新式はそのまま再利用し、計算済みの加速度を次のステップへ渡しました。多数粒子を置いても、粒子同士の重力は含めていません。

数値 CDF の誤差、有限粒子数のばらつき、軌道積分の誤差は、別の比較で確認しました。円盤の形が保たれることに加えて、各粒子の保存量と、同時刻の解析位置を調べました。出力を間引くことと、時間刻みを大きくすることも区別しました。

第 5 章では、この円盤を二つ用意します。第 2 章の銀河中心の運動、第 3 章の Plummer 重力、第 4 章の円盤初期条件を組み合わせ、互いに接近させます。今回確かめた孤立円盤を基準にすることで、相互作用による変化と、初期条件や数値誤差による変化を区別しやすくなります。

演習

演習 4.1　確認：面密度と半径の分布　$R_d = 3\,\mathrm{kpc}$、$R_{\max} = 5R_d$ について、$p_R(R)$ の規格化を確認してください。$R < R_d$ の粒子割合を求め、$N = 1000$ なら平均的に何個入るか答えてください。$p_R \propto e^{-R/R_d}$ とした場合に、何を落としているかも説明してください。

演習 4.2　確認：線形補間と逆補間　表 $(x_j, y_j) = (0,0), (1,2), (3,3)$ に対して、$x = 2$ の y を自作関数と `np.interp` で求めてください。次に $y = 2.5$ に対応する x を、表の入力側と出力側を交換して求めてください。$y = 4$ を渡す場合は、本章の自作関数と `np.interp` の既定の扱いにどのような違いがありますか。

演習 4.3　確認：有限範囲の規格化と中央値　CDF 表の最後の値を 1 にすることが、第 3 章の Plummer 累積質量を無条件に全質量へそろえることと異なる理由を述べてください。$F(R) = 0.5$ を数値表の逆補間で解き、解析 CDF を既習の二分法で解いた値と比較してください。両者の差と、実際に生成した 1000 粒子の半径の中央値のずれは、同じ原因でしょうか。

演習 4.4　応用：粒子数と乱数 seed　表4.3と同じ N、seed で経験 CDF のずれを計算してください。N を変える実験では、$M, a, R_d, R_{\max}, N_R$ を固定します。粒子数を増やしても 1 粒子の軌道計算の次数が上がるわけではない理由を説明してください。

演習 4.5　応用：時間刻みと保存間隔　まず同じ初期配列で時間刻みを変え、KDK の 2 次精度を確認してください。次に時間刻みを固定して、`save_every=1,10,50` を比べてください。各計算の最終配列、保存時刻数、履歴の容量はどうなりますか。粗い出力で求めた最大誤差を、全ステップの最大誤差と呼べるでしょうか。

演習 4.6　応用：回転方向　同じ seed・同じ粒子数で $s_{\mathrm{rot}} = +1, -1$ の初期条件を作ってください。位置が一致し、速度だけが符号反転すること、初期比エネルギーが一致し、比角運動量の符号が反転することを確認してください。両方を KDK で進め、それぞれに対応する円運動の解析位置と比較してください。

演習 4.7　発展：円運動速度の 80% で始める　初期位置を標準円盤と同じに保ち、初期速度だけを 0.8 倍してください。半径分布の時間変化と各粒子の保存量を調べてください。この実験を自己重力円盤の不安定性と呼んでよいでしょうか。また、標準の円運動の解析位置との差を、そのまま数値誤差と呼べない理由を説明してください。